암호학적 배경 및 보안 고려사항

개요

다항식 커밋먼트 스킴은 영지식 증명 시스템의 핵심 구성 요소로, 각각 서로 다른 암호학적 가정과 보안 특성을 가지고 있습니다. 이 문서는 arkworks-rs/poly-commit 라이브러리에서 구현된 각 스킴의 수학적 기반, 보안 가정, 그리고 실제 운영 환경에서 고려해야 할 사항들을 다룹니다.

수학적 기반과 암호학적 가정

1. KZG10 스킴

수학적 기반:

- **타원곡선 페어링(Elliptic Curve Pairing)**: KZG10은 bilinear pairing `e: G1 × G2 → GT` 를 사용합니다
- **다항식 나눗셈**: witness polynomial은 `w(x) = (p(x) - p(z)) / (x - z)` 로 계산됩니다
- **MSM(Multi-Scalar Multiplication)**: 커밋먼트 생성 시 효율적인 다중 스칼라 곱셈을 사용합니다

암호학적 가정:

- **q-Strong Diffie-Hellman (q-SDH) 가정**: 주어진 `(g, g^α, g^(α²), ..., g^(α^q))` 에서 `(c, g^(1/(α+c)))` 를 계산하는 것이 어렵다는 가정
- **Discrete Logarithm Problem**: 타원곡선 상에서 이산 로그 문제의 어려움
- **Knowledge of Exponent (KoE) 가정**: Algebraic Group Model에서의 extractability를 보장

보안 특성:

- **Perfect Completeness**: 정직한 prover는 항상 유효한 증명을 생성할 수 있습니다

- **Computational Soundness**: q-SDH 가정 하에서 거짓 증명을 생성하는 것은 계산적으로 불가능합니다
- **Zero-Knowledge**: hiding commitment 사용 시 다항식에 대한 정보를 누출하지 않습니다

```
// KZG10의 핵심 보안 가정을 보여주는 setup 과정
pub fn setup<R: RngCore>(
max_degree: usize,
produce_g2_powers: bool,
rng: &mut R,
) -> Result<UniversalParams<E>, Error> {
let beta = E::ScalarField::rand(rng); // 비밀 값 β
let g = E::G1::rand(rng);
let h = E::G2::rand(rng);
// powers_of_beta = [1, β, β², ..., β^max_degree]
// 이 값들이 노출되면 안 되므로 trusted setup이 필요
let mut powers_of_beta = vec![E::ScalarField::one()];
let mut cur = beta;
for _ in 0..=max_degree {
powers_of_beta.push(cur);
cur *= &beta;
}
// G1에서의 powers: [g, g^β, g^β², ...]
let powers_of_g = g.batch_mul(&powers_of_beta[0..max_degree + 1]);
// ...
```

```
}
```

2. IPA (Inner Product Argument) 스킴

수학적 기반:

- **Inner Product Relations**: 벡터의 내적 관계를 이용한 증명
- **Pedersen Commitment**: `Com(v, r) = g^v · h^r` 형태의 커밋먼트
- **Recursive Halving**: 로그 크기의 증명을 위한 재귀적 축소

암호학적 가정:

- **Discrete Logarithm Problem**: 소수 차수 군에서의 이산 로그 문제
- **No Trusted Setup**: 신뢰할 수 있는 설정이 불필요합니다

보안 특성:

- **Transparent Setup**: 공개적으로 검증 가능한 설정 과정
- **Post-Quantum Resistance**: 양자 컴퓨터에 대한 저항성은 제한적
- **Logarithmic Proof Size**: O(log n) 크기의 증명

```
// IPA의 투명한 설정 과정

fn sample_generators(num_generators: usize) -> Vec<G> {

let generators: Vec<_> = ark_std::cfg_into_iter!(0..num_generators)

.map(|i| {

let i = i as u64;

// 결정론적으로 생성되는 generator들

let mut hash = D::digest([Self::PROTOCOL_NAME, &i.to_le_bytes()].concat().as_sli

let mut g = G::from_random_bytes(&hash);
```

```
// ...
})
.collect();
generators
}
```

3. Hyrax 스킴

수학적 기반:

- **Multilinear Extension**: 다변수 다항식의 multilinear extension
- **Matrix Representation**: 다항식을 행렬로 표현하여 커밋
- **Tensor Product**: 평가점을 텐서곱으로 분해

암호학적 가정:

- **Discrete Logarithm Problem**: 타원곡선 군에서의 이산 로그 문제
- **Square Matrix Requirement**: 변수 개수가 짝수여야 함

보안 특성:

- **Square-root Commitment Size**: O(√n) 크기의 커밋먼트
- **Interactive Protocol**: Fiat-Shamir 변환으로 non-interactive화

4. Linear Codes 기반 스킴 (Ligero, Brakedown)

수학적 기반:

- **Error-Correcting Codes**: 오류 정정 부호를 이용한 커밋먼트
- **Reed-Solomon Codes**: 다항식 평가를 이용한 인코딩
- **Merkle Tree**: 커밋먼트의 무결성 보장

암호학적 가정:

- **Hash Function Security**: 충돌 저항성과 일방향성
- **No Algebraic Structure**: 대수적 구조에 의존하지 않음

보안 특성:

- **Post-Quantum Security**: 양자 컴퓨터에 대한 강한 저항성
- **Concrete Security**: 구체적인 보안 매개변수 설정 가능

Universal Setup과 Trusted Ceremony

Trusted Setup이 필요한 스킴

KZG10과 Marlin PC:

```
// 위험한 비밀 값들이 setup에서 사용됨
let beta = E::ScalarField::rand(rng); // 이 값이 노출되면 soundness 깨짐
let gamma = E::ScalarField::rand(rng); // hiding을 위한 추가 비밀값


// 이 값들은 setup 후 반드시 삭제되어야 함
// "toxic waste" - 보관 시 전체 시스템의 보안이 위험
```

Trusted Setup의 중요성:

1. **Soundness 보장**: 비밀 값이 노출되면 거짓 증명 생성 가능
2. **Universal Property**: 한 번의 setup으로 모든 다항식에 사용 가능
3. **Ceremony 필요성**: 다중 참여자를 통한 분산 생성 필요

Powers of Tau Ceremony:

```
Phase 1: Universal setup for all circuits
- 참여자들이 순차적으로 비밀값 기여
- 한 명이라도 정직하면 전체 ceremony 안전
- 최종 결과물: structured reference string (SRS)


Phase 2: Circuit-specific setup (필요한 경우)
- 특정 회로에 대한 추가 설정
- KZG10은 universal하므로 Phase 1만 필요
```

Transparent Setup 스킴

IPA, Hyrax, Linear Codes:

- 공개적으로 검증 가능한 설정
- 신뢰할 수 있는 제3자 불필요
- 결정론적 generator 생성

Production 환경에서의 보안 고려사항

1. 매개변수 선택

보안 수준 설정:

```rust
// 128비트 보안을 위한 권장 매개변수
const SECURITY_LEVEL: usize = 128;
const FIELD_SIZE_BITS: usize = 256; // BLS12-381 scalar field
```

```
const CURVE_SECURITY: usize = 128; // 타원곡선 보안 수준


// Linear codes의 경우 구체적 보안 매개변수
let sec_param = 128; // λ = 128
let distance = (rate_num, rate_den); // 부호의 거리
```

Field 크기 고려사항:

- **BLS12-381**: 128비트 보안, pairing-friendly
- **BN254**: 100비트 보안 (더 이상 권장되지 않음)
- **Scalar field 크기**: 2^k - 1 형태의 소수 선호

2. 구현 보안

Side-channel 공격 방어:

```
// 상수 시간 구현 필요
use ark_ff::Field;


// 잘못된 예: 분기가 비밀값에 의존
if secret_bit == 1 {
result = base * multiplier;
} else {
result = base;
}
```

```
// 올바른 예: 상수 시간 조건부 선택

let multiplier = Field::conditional_select(&Field::one(), &multiplier, secret_bi

result = base * multiplier;
```

메모리 보안:

```
use zeroize::Zeroize;


struct SecretState {

secret_key: Vec<u8>,

randomness: Vec<u8>,

}


impl Drop for SecretState {

fn drop(&mut self) {

self.secret_key.zeroize();

self.randomness.zeroize();

}

}
```

3. 난수 생성

암호학적으로 안전한 RNG 사용:

```
use ark_std::rand::{CryptoRng, RngCore};
```

```rust
use rand_chacha::ChaCha20Rng;

use rand_core::SeedableRng;


// 시스템 엔트로피 사용

let mut rng = ChaCha20Rng::from_entropy();


// 또는 시드 기반 (재현 가능한 테스트용)

let seed = [0u8; 32]; // 실제로는 안전한 시드 사용

let mut rng = ChaCha20Rng::from_seed(seed);


// 커밋먼트 생성 시 반드시 RNG 제공

let (commitment, randomness) = PC::commit(&ck, &polynomials, Some(&mut rng))?;
```

4. 배치 검증 최적화

보안과 성능의 균형:

```rust
// 배치 검증 시 randomizer 사용

pub fn batch_check<R: RngCore>(

vk: &VerifierKey<E>,

commitments: &[Commitment<E>],

points: &[E::ScalarField],

values: &[E::ScalarField],

proofs: &[Proof<E>],
```

```
rng: &mut R, // 배치 검증을 위한 randomness

) -> Result<bool, Error> {

let mut randomizer = E::ScalarField::one();

for (commitment, proof) in commitments.iter().zip(proofs) {

// 각 증명에 대해 다른 randomizer 적용

// 이를 통해 개별 증명의 실패를 숨김

randomizer = u128::rand(rng).into();

// ...

}

}
```

알려진 제한사항과 주의사항

1. KZG10 제한사항

Trusted Setup 의존성:

- Setup ceremony의 무결성에 전적으로 의존
- 비밀값 노출 시 전체 시스템 compromise
- Universal setup이지만 최대 차수 제한 존재

성능 고려사항:

```
// 큰 차수의 다항식은 setup 시간과 메모리 사용량 증가

let max_degree = 2_usize.pow(20); // 2^20 = ~100만

let pp = KZG10::setup(max_degree, false, rng)?; // 메모리 집약적
```

```
// Pairing 연산은 상대적으로 비용이 높음
let result = E::pairing(commitment, witness); // 약 1-2ms
```

2. IPA 제한사항

증명 크기와 검증 시간:

- 증명 크기: O(log n), 하지만 상수가 큼
- 검증 시간: O(n), 큰 다항식에서 비효율적
- Pairing 불필요하지만 많은 지수 연산 필요

3. Linear Codes 제한사항

구체적 보안 분석 필요:

```
// 보안 매개변수 계산이 복잡
fn calculate_security_level(
field_size: usize,
code_rate: (usize, usize),
num_queries: usize,
) -> usize {
// Johnson bound, Plotkin bound 등을 고려한 계산
// 실제 구현에서는 전문가의 분석 필요
}
```

Field 크기 제약:

- 작은 field에서는 보안 수준 달성 어려움
- 큰 field는 통신 비용 증가

4. 일반적 주의사항

Fiat-Shamir 변환:

```
// 해시 함수의 선택이 중요
use blake2::Blake2s256; // 권장
// use sha2::Sha256; // 가능하지만 느림
// use md5::Md5; // 절대 사용 금지


// Challenge 생성 시 충분한 엔트로피 확보
sponge.absorb(&commitment_bytes);
sponge.absorb(&point_bytes);
sponge.absorb(&additional_context); // 재사용 공격 방지
let challenge = sponge.squeeze_field_elements(1)[0];
```

Degree Bound 강제:

```
// Degree bound 검증 누락 시 soundness 문제
if let Some(degree_bound) = polynomial.degree_bound() {
if polynomial.degree() > degree_bound {
return Err(Error::IncorrectDegreeBound { /* ... */ });
}
// Shifted commitment를 통한 강제 필요
```

```
}
```

실무 권장사항

1. 스킴 선택 기준

KZG10 사용 권장:

- Proof 크기가 중요한 경우 (상수 크기)
- Pairing-friendly curve 사용 가능한 환경
- Trusted setup 수용 가능한 경우

IPA 사용 권장:

- Trusted setup 회피 필요한 경우
- 투명성이 중요한 공개 시스템
- 중간 크기의 다항식 (degree < 2^16)

Linear Codes 사용 권장:

- Post-quantum 보안 필요한 경우
- 매우 큰 다항식 처리
- Hash 기반 보안 선호

2. 보안 감사 체크리스트

Setup 단계:

- [ ] 적절한 보안 매개변수 선택
- [ ] 신뢰할 수 있는 난수 생성기 사용
- [ ] Trusted setup의 경우 ceremony 검증

구현 단계:

- [ ] 상수 시간 구현 확인
- [ ] 메모리 누수 방지
- [ ] Side-channel 공격 대응

운영 단계:

- [ ] 정기적인 보안 업데이트
- [ ] 모니터링 및 로깅 시스템
- [ ] 인시던트 대응 계획

3. 성능 최적화

배치 처리 활용:

```
// 개별 검증 대신 배치 검증 사용
let results = commitments.chunks(BATCH_SIZE)
.map(|batch| PC::batch_check(vk, batch, points, values, proofs, rng))
.collect::<Result<Vec<_>, _>>()?;
```

메모리 관리:

```
// 큰 다항식 처리 시 스트리밍 방식 고려
use ark_poly_commit::streaming_kzg::StreamingKZG;

// 또는 메모리 매핑 파일 사용
use memmap2::MmapOptions;
```

이러한 고려사항들을 통해 안전하고 효율적인 다항식 커밋먼트 시스템을 구축할 수 있습니다. 특히 production 환경에서는 보안 전문가의 검토와 지속적인 모니터링이 필수적입니다.

관련 문서

기본 이해

- **프로젝트 개요**: 각 스킴의 기본 특징과 보안 가정 개요
- **핵심 인터페이스**: 보안 관련 에러 처리와 데이터 구조의 설계 원칙

구현 분석

- **KZG10 구현**: KZG10의 trusted setup과 구체적 보안 구현
- **Marlin PC 구현**: Degree bound 강제의 보안적 의미
- **기타 스킴들**: 각 스킴의 보안 가정과 특성 비교

실무 적용

- **사용 예제**: 보안 매개변수 설정과 안전한 사용법 예제
- **용어집**: 보안 관련 용어와 수학적 가정 정리